Oxford Cambridge and RSA – style practice paper

GCSE (9–1) Computer Science

J277/02 Computational thinking, algorithms and programming

Topic 2.1: Algorithms

Candidate forename		**Candidate surname**	
Centre number		**Candidate number**	

Practice questions in the style of OCR GCSE Computer Science (J277). Every question is original; the layout, command words, mark allocations and mark scheme follow the real exam format.

Time allowed	About 110 minutes (roughly 1 minute per mark)
Instructions	• Use black ink. HB pencil may be used for graphs and diagrams only. • Answer **all** the questions. • Write your answer to each question in the space provided. • Where the question asks you to write code or pseudocode, you may use any suitable high-level language or the OCR Reference Language; marks are for the logic, not exact syntax. • Do **not** use a calculator or any other electronic device unless told to.
Information	• The total mark for this paper is **100**. • The marks for each question are shown in brackets [ ]. • Quality of extended responses will be assessed in questions marked with an asterisk (*). • Where a question asks you to *state* or *give*, a short answer is enough; *describe* and *explain* need more detail.

Name: _____ Date: _____ Mark: _____ / 100

1 Computational thinking is used to solve problems.

(a) Draw **one** line from each term to its correct description. **[3]**

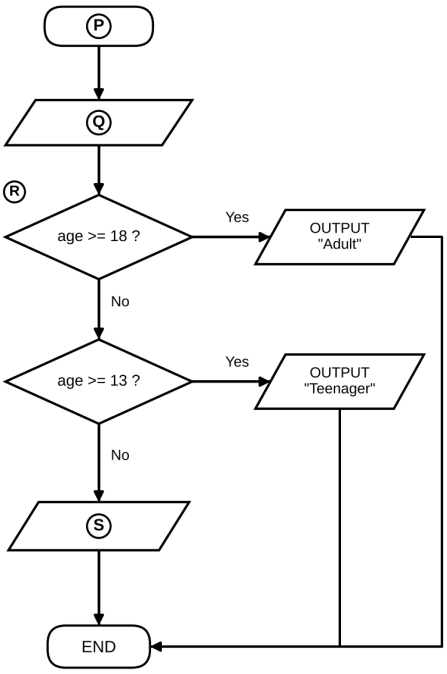
Term	Description
Abstraction	Breaking a problem down into smaller, easier-to-solve parts
Decomposition	Writing a step-by-step set of instructions to solve a problem
Algorithmic thinking	Removing unnecessary detail so that only the important parts remain

(b) A map of the London Underground shows stations as dots and routes as coloured lines. It does not show the exact distances between stations. Explain how this is an example of abstraction. **[2]**

(c) A programmer is creating a quiz game. Describe how **decomposition** could be used when planning the program. **[3]**

(d) State **one** benefit of decomposing a problem. **[1]**

2 The flowchart shows an algorithm. Symbols **P**, **Q**, **R** and **S** are not labelled.



(a) Name the symbol labelled **P**. **[1]**

(b) Name the symbol labelled **R** and state what it is used for. **[2]**

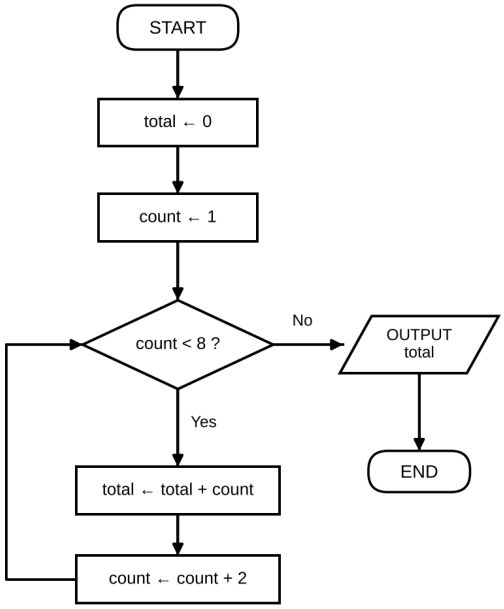
(c) Write down what should go in the symbol labelled **Q**. **[1]**

(d) State the output of the algorithm when the input is 15. **[1]**

(e) The user enters 9. Write the text that should be in symbol **S**. **[1]**

(f) State why the algorithm outputs the same result for an input of 18 as for an input of 90. **[1]**

3 The flowchart shows a different algorithm.



(a) Complete the trace table for the algorithm. **[4]**

total	**count**	**count < 8 ?**
0	1	Yes

(b) State the value that is output. **[1]**

(c) Describe what the algorithm does. **[1]**

(d) The value of 8 in the decision box is changed to 12. State the new output. **[1]**

4 A program contains the following algorithm, written in the OCR Reference Language.

(a) State the output when the program is run. **[1]**

```
total = 0
for i = 1 to 5
    total = total + i
next i
print(total)
```

(b) Complete the trace table for this algorithm. **[3]**

i	**total**

(c) Change the algorithm so that it adds up the numbers from 1 to 10. Write the new line(s) of code you would change. **[1]**

(d) Rewrite the algorithm using a **while** loop that produces the same output. **[4]**

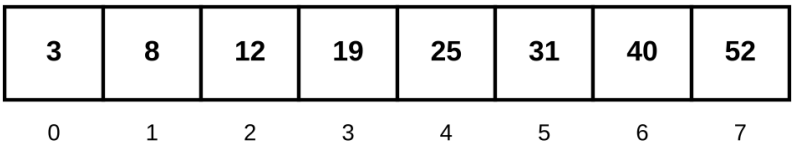
5 Searching algorithms are used to find items in a list.

(a) Describe how a **linear search** finds an item in a list. **[3]**

(b) A list contains 1000 items. State the **maximum** number of items a linear search must check. **[1]**

(c) Give **one** advantage of a linear search over a binary search. **[1]**

(d) The list of numbers is stored in an array, shown in the diagram. A binary search is used to find the value **31**. State the value of the middle item first checked. Use the integer midpoint (low + high) DIV 2. **[2]**



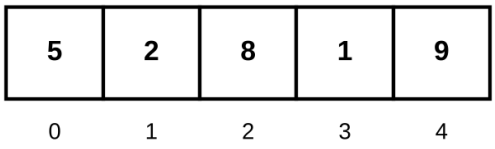
(e) Complete the trace table for the binary search for 31. The first row has been completed. **[3]**

low	high	mid	list[mid]	Result
0	7	3	19	19 < 31 so search upper half

(f) Show that the value **10** is not in the list by using a binary search. Show the middle values you check each time. **[3]**

(g) Explain why a binary search cannot be used on an unsorted list. **[2]**

6 A bubble sort is used to sort the list **5, 2, 8, 1, 9** into ascending order.



(a) Describe how a bubble sort works. **[3]**

(b) Write the list after the first pass of the bubble sort. **[2]**

(c) State the number of swaps made in the first pass. **[1]**

(d) Write the list after the second pass. **[1]**

(e) The list is completely sorted after the third pass. Explain why a fourth pass is still needed. **[2]**

7 Merge sort and insertion sort are also sorting algorithms.

(a) The list **7, 3, 9, 1, 5, 2, 8, 4** is sorted using a merge sort. Complete the diagram to show the merging stages. **[4]**

Stage	**Lists**
Split to single items	[7] [3] [9] [1] [5] [2] [8] [4]
Merge into pairs	
Merge into fours	
Final merge	

(b) Describe the first stage of a merge sort. **[2]**

(c) An insertion sort is used on the list **6, 3, 5, 1**. Complete the table to show the list after each item has been inserted into the sorted part. **[3]**

After inserting	**List**
start	6, 3, 5, 1
3	
5	
1	

(d) Give **one** advantage of merge sort over bubble sort. **[1]**

(e) Give **one** disadvantage of merge sort compared to bubble sort. **[1]**

8 A teacher has written an algorithm to find the largest of three numbers.

(a) The algorithm contains a logic error. Identify the error and correct it. **[2]**

```
a = input("Enter first number")
b = input("Enter second number")
c = input("Enter third number")
largest = a
if b > largest then
    largest = b
endif
if b > largest then
    largest = c
endif
print(largest)
```

(b) Give the output of the original code when the user enters 4, 9 and 20. **[2]**

(c) State **one** test input that would show the code is incorrect. **[2]**

9 Writing algorithms.

(a) Write an algorithm, using pseudocode or a flowchart, that asks the user to enter 10 numbers and then outputs the average. **[5]**

(b) A user is allowed three attempts at entering a password. Write an algorithm that asks for a password up to three times and outputs "Access granted" if it matches the stored password or "Locked" after three wrong attempts. **[6]**

10 Extended response questions.

(a) *Compare the linear search and binary search algorithms. In your answer you should describe how each works, and discuss when each would be the best choice. **[8]**

(b) *Discuss the advantages and disadvantages of bubble sort, insertion sort and merge sort. **[8]**

END OF QUESTIONS

MARK SCHEME – Topic 2.1: Algorithms

How to use this mark scheme: mark your own work in a different colour. A bullet (•) in the Answer column is a separate marking point worth one mark unless stated. A slash (/) separates alternative answers that are equally acceptable. Words in brackets are not needed. **Accept** = acceptable; **Ignore** = neither credited nor penalised; **Do not accept** = contradicts or spoils the answer. For programming questions, any valid solution that works is acceptable and syntax errors are not penalised if the intent is clear. The code in the answers uses the OCR Reference Language style.

Question	Answer	Marks	Guidance
1(a)	• abstraction: removing unnecessary detail to focus on the important parts ; • decomposition: breaking a problem into smaller sub-problems ; • algorithmic thinking: creating a step-by-step set of instructions to solve a problem ;	3 AO1 2.1.1	One mark per correct match.
1(b)	• unnecessary detail (real distances / street layout) is removed ; • only the information the passenger needs (stations and connections) is kept ;	2 AO2 2.1.1	
1(c)	• split the game into smaller parts / sub-problems ; • e.g. asking questions, checking answers, keeping score, displaying results ; • each part can be solved / programmed / tested separately (by different programmers) ;	3 AO2 2.1.1	
1(d)	easier to solve / test / maintain / work on in a team ;	1 AO1 2.1.1	
2(a)	terminator / start (terminal) ;	1 AO1 2.1.2	
2(b)	• decision ; • it asks a question with two (yes / no) outcomes / used for selection ;	2 AO1 2.1.2	
2(c)	INPUT age / get the user to enter their age ;	1 AO2 2.1.2	
2(d)	"Teenager" ;	1 AO2 2.1.2	
2(e)	OUTPUT "Child" ;	1 AO2 2.1.2	Accept any suitable wording e.g. Child.
2(f)	both satisfy age >= 18 (so both take the 'Yes' branch) ;	1 AO3 2.1.2	
3(a)	• total: 0, 1, 4, 9, 16 (correct totals column) ; • count: 1, 3, 5, 7, 9 (correct count column) ; • count < 8 column: Yes ×4 then No ; • rows in the correct order ;	4 AO2 2.1.3	Final table: (0, 1, Yes), (1, 3, Yes), (4, 5, Yes), (9, 7, Yes), (16, 9, No). Award marks per column with errors carried forward.
3(b)	16 ;	1 AO2 2.1.3	
3(c)	adds up the odd numbers less than 8 (1 + 3 + 5 + 7) / calculates the sum of odd numbers ;	1 AO3 2.1.3	
3(d)	36 ;	1 AO3 2.1.3	1 + 3 + 5 + 7 + 9 + 11 = 36.
4(a)	15 ;	1 AO2 2.1.3	

Question	Answer	Marks	Guidance
4(b)	• i column: 1 2 3 4 5 ; • total column: 1 3 6 10 15 ; • correct rows ;	3 AO2 2.1.3	
4(c)	for i = 1 to 10 ;	1 AO2 2.1.3	
4(d)	• total = 0 / i = 1 initialised ; • while i <= 5 ; • total = total + i and i = i + 1 inside the loop ; • endwhile / print(total) outside loop ;	4 AO3 2.1.3	Example: `total = 0`, `i = 1`, `while i <= 5`, `total = total + i`, `i = i + 1`, `endwhile`, `print(total)`.
5(a)	• start at the first item ; • compare each item with the one being searched for ; • stop when found or when the end of the list is reached ;	3 AO1 2.1.4	
5(b)	1000 ;	1 AO2 2.1.4	
5(c)	• the list does not need to be sorted ; • simpler to code ;	1 AO1 2.1.4	Any one.
5(d)	• (0 + 7) DIV 2 = 3 ; • 19 ;	2 AO2 2.1.4	
5(e)	• row 2: low 4, high 7, mid 5, item 31 ; • found / compare 31 = 31 ; • (check both) outcome: found at index 5 ;	3 AO2 2.1.4	low = 4, high = 7, mid = 5, list[5] = 31 → found.
5(f)	• mid = 3 (19 > 10) so high = 2 ; • mid = 1 (8 < 10) so low = 2, then mid = 2 (12 > 10) so high = 1 ; • low > high so not found ;	3 AO3 2.1.4	
5(g)	• binary search discards half of the list each time by comparing with the middle value ; • this only works if the data is in order, otherwise the item may be in the discarded half ;	2 AO2 2.1.4	
6(a)	• compare adjacent pairs of items ; • swap them if they are in the wrong order ; • repeat passes through the list until no swaps are made ;	3 AO1 2.1.5	
6(b)	• 2, 5, 1, 8, 9 ; • (1 mark for 4 correct items in their positions) ;	2 AO2 2.1.5	Pass 1: 5↔2, 5 vs 8 no swap, 8↔1, 8 vs 9 no swap.
6(c)	2 ;	1 AO2 2.1.5	
6(d)	2, 1, 5, 8, 9 ;	1 AO2 2.1.5	
6(e)	• the algorithm only knows the list is sorted when a pass has no swaps ; • so another pass is needed to check there are no swaps ;	2 AO3 2.1.5	
7(a)	• after first merges: [3,7], [1,9], [2,5], [4,8] ; • second merges: [1,3,7,9], [2,4,5,8] ; • final: [1,2,3,4,5,7,8,9] ; • all correct order within each group ;	4 AO2 2.1.5	
7(b)	• the list is repeatedly divided in half ; • until each sub-list has only one item ;	2 AO1 2.1.5	

Question	Answer	Marks	Guidance
7(c)	• 3, 6, 5, 1 ; • 3, 5, 6, 1 ; • 1, 3, 5, 6 ;	3 AO2 2.1.5	
7(d)	faster for large lists / fewer comparisons ;	1 AO1 2.1.5	
7(e)	• uses more memory (extra lists needed) ; • more complex to code ;	1 AO1 2.1.5	Any one.
8(a)	• the second if compares b with a instead of c / uses b > c incorrectly (any valid error identification) ; • corrected: elseif c > a then largest = c (or equivalent) ;	2 AO3 2.1.3	The second condition should read if c > largest then.
8(b)	• largest = 9 after first if ; • second if: 9 > 9 False so largest stays 9 → outputs 9 ;	2 AO2 2.1.3	
8(c)	• any input where c is the largest, e.g. 4, 9, 20 ; • expected output 20 but the program outputs 9 ;	2 AO3 2.1.3	Accept any set of three numbers with c largest.
9(a)	• total initialised to 0 ; • loop repeated 10 times ; • input a number and add it to total inside the loop ; • calculate total / 10 ; • output the average ;	5 AO3 2.1.2	Example: total = 0 for i = 1 to 10 n = input("Enter a number") total = total + n next i print(total / 10)
9(b)	• stored password / attempts initialised ; • loop with up to three attempts ; • input password in the loop ; • compare with stored password ; • output "Access granted" and stop if correct ; • output "Locked" if all three attempts fail ;	6 AO3 2.1.2	Any logically correct solution: for or while loops are acceptable. Do not penalise minor syntax errors.

Question	Answer	Marks	Guidance
10(a)	**Level 3 (high), 6–8 marks**: Thorough knowledge and understanding of the topic with detailed, accurate and relevant points. A balanced, well-structured answer reaching a justified conclusion where asked. Technical terms used correctly throughout. **Level 2 (mid), 3–5 marks**: Reasonable knowledge and understanding. Some relevant points made, but the answer lacks detail, balance or a clear structure in places. **Level 1 (low), 1–2 marks**: Basic knowledge. A few simple, mostly isolated points with little or no development or linking. **Level 0, 0 marks**: No response or no response worthy of credit. **Indicative content** (credit any other valid points): • linear: starts at the first item and checks each in turn until found / end of list; works on unsorted data • binary: list must be sorted; checks the middle item and discards half the list each time (halves the search area) • binary needs far fewer comparisons for large lists (e.g. 1000 items needs ~10 checks vs up to 1000) • linear is simple to code; suitable for small lists or when the data is unsorted • if the data is not sorted, binary search requires sorting first, which takes time • linear might find an item immediately if it is near the start; worst case is the whole list • balanced conclusion: choose based on list size, whether sorted and how often the list will be searched	8 AO1 / AO2 / AO3 2.1.4	
10(b)	**Level 3 (high), 6–8 marks**: Thorough knowledge and understanding of the topic with detailed, accurate and relevant points. A balanced, well-structured answer reaching a justified conclusion where asked. Technical terms used correctly throughout. **Level 2 (mid), 3–5 marks**: Reasonable knowledge and understanding. Some relevant points made, but the answer lacks detail, balance or a clear structure in places. **Level 1 (low), 1–2 marks**: Basic knowledge. A few simple, mostly isolated points with little or no development or linking. **Level 0, 0 marks**: No response or no response worthy of credit. **Indicative content** (credit any other valid points): • bubble sort: compares adjacent items and swaps; simple and easy to code and understand; inefficient (many passes) for large lists • insertion sort: takes each item and inserts it into the correct place in the sorted part; fairly simple; efficient for small or nearly sorted lists; slow for large lists • merge sort: divides the list into single items and merges back in order (divide and conquer); much faster for large lists • merge sort needs more memory for the extra lists and is harder to code • bubble sort and insertion sort sort in place (little extra memory) • choice depends on the size of the list, how sorted the data already is, memory available and how much programming time there is • balanced conclusion	8 AO1 / AO2 / AO3 2.1.5	

Where marks are lost in Topic 2.1

Use this page after marking. Tick the ones that cost you marks, then redo those questions closed-book.

- ☐ **Trace tables:** update only the variable that changes on each line and check the loop condition every time. Don't skip the final row where the loop ends.
- ☐ **Binary search:** mid = (low + high) DIV 2. If target > list[mid] then low = mid + 1, otherwise high = mid − 1. Stop when low > high (not found).
- ☐ **Binary search needs sorted data.** Linear search does not. This is the most-tested comparison.
- ☐ **Bubble sort:** compare *adjacent* items; the biggest value 'bubbles' to the end each pass; stop when a pass makes no swaps.
- ☐ **Merge sort:** divide to single items, then merge in order. Mention 'divide and conquer' and that it is faster for big lists but uses more memory.
- ☐ **Flowchart symbols:** rounded = start/end, rectangle = process, parallelogram = input/output, diamond = decision (two exits labelled Yes / No).
- ☐ **Writing algorithms:** initialise before the loop, update inside it, output after it. Many marks are lost for forgetting `total = 0`.
- ☐ **Abstraction vs decomposition:** abstraction = remove detail; decomposition = split into parts.